Java Backend Interview Series

Sub-Chapter 1.10: Java 9–21 Features (Modern Java)

Java 9 · Java 10 · Java 11 · Java 14 · Java 16 · Java 17 · Java 21

50+ Q&A • Advanced Questions • Exercises • Cheat Sheet

1. Java 9: Module System (JPMS) & JShell

Java 9 introduced the **Java Platform Module System (JPMS)** — Project Jigsaw — enforcing strong encapsulation and reliable configuration. A module declares its dependencies and exports via **module-info.java**. JShell provides an interactive REPL.

```
// module-info.java
module com.myapp.service {
requires java.sql;
requires transitive com.myapp.model;
exports com.myapp.service.api;
opens com.myapp.service.impl to spring.core;
uses com.myapp.service.api.Plugin;
provides com.myapp.service.api.Plugin
with com.myapp.service.impl.DefaultPlugin;
}
```

Directive	Purpose
requires	Declares module dependency
requires transitive	Implies dependency to downstream modules
exports	Makes package accessible to all modules
exports … to	Makes package accessible to specific modules
opens	Allows deep reflection at runtime (e.g., Spring)
uses / provides	ServiceLoader integration (SPI)

Key Java 9 Additions

Factory methods for Collections: List.of(), Set.of(), Map.of(), Map.ofEntries(). Stream.takeWhile(), dropWhile(), iterate(seed, predicate, f). Optional.ifPresentOrElse(), stream(). Private methods in interfaces. Process API improvements. HTTP/2 Client (incubator).

2. Java 10–11: var Keyword & HTTP Client

Java 10 introduced **local-variable type inference** with the **var** keyword. It is not dynamic typing — the compiler infers the static type at compile time. **Java 11** made the HTTP Client standard, added String utilities, and removed Java EE/CORBA modules.

```
// var — type inferred at compile time
var list = new ArrayList<String>(); // ArrayList<String>
var map = new HashMap<String, Integer>();
var path = Path.of("/tmp/data.txt");
// var in lambda parameters (Java 11)
list.stream()
.filter((@NonNull var s) -> s.startsWith("A"))
.forEach(System.out::println);
// Java 11 String methods
" hello ".strip(); // Unicode-aware trim
"".isBlank(); // true
"line1
line2".lines().count(); // 2
"abc".repeat(3); // "abcabcabc"
// Java 11 HTTP Client
HttpClient client = HttpClient.newBuilder()
.version(HttpClient.Version.HTTP_2)
.connectTimeout(Duration.ofSeconds(5))
```

```
.build();
HttpRequest req = HttpRequest.newBuilder()
.uri(URI.create("https://api.example.com/data"))
.GET().build();
HttpResponse<String> resp =
client.send(req, BodyHandlers.ofString());
System.out.println(resp.body());
```

3. Java 14–16: Records, Pattern Matching, Sealed Classes

Records (finalized in Java 16) are immutable data carriers that automatically generate constructor, accessors, equals, hashCode, and toString. **Pattern matching for instanceof** eliminates redundant casts. **Sealed classes** restrict which classes can extend or implement a type.

```
// Record — immutable data carrier
record Point(double x, double y) {
// compact canonical constructor
Point {
if (x < 0 || y < 0) throw new IllegalArgumentException("Negative coords");
}
// custom method
double distance() { return Math.sqrt(x*x + y*y); }
}
// Pattern Matching instanceof (Java 16)
Object obj = "Hello, Java 16!";
if (obj instanceof String s && s.length() > 5) {
System.out.println(s.toUpperCase());
}
// Sealed Classes (Java 17)
public sealed interface Shape
permits Circle, Rectangle, Triangle {}
public record Circle(double radius) implements Shape {}
public record Rectangle(double w, double h) implements Shape {}
public final class Triangle implements Shape { /* ... */ }
// Switch expression with pattern (Java 21)
double area = switch (shape) {
case Circle c -> Math.PI * c.radius() * c.radius();
case Rectangle r -> r.w() * r.h();
case Triangle t -> 0.5 * t.base() * t.height();
};
```

Records Constraints

Records are implicitly final. All fields are private final. Cannot extend other classes. Can implement interfaces. Can have static fields/methods. Compact constructors validate without assigning — assignments happen automatically after the body.

4. Java 15–17: Text Blocks & Sealed Types (LTS)

Text Blocks (finalized Java 15) allow multi-line string literals with managed indentation. Java 17 is a **Long-Term Support (LTS)** release featuring sealed classes, pattern matching instanceof, enhanced pseudo-random generators (PRNG), and removal of SecurityManager.

```
// Text Block — manages indentation automatically
String json = """
{
"name": "Alice",
"age": 30,
```

```
"active": true
}
""";
String html = """
<html>
<body><p>Hello %s</p></body>
</html>
""".formatted("World");
// Text block methods
String stripped = """
line1
line2
""".stripIndent(); // removes common leading whitespace
// Enhanced PRNG (Java 17)
RandomGeneratorFactory.all()
.map(RandomGeneratorFactory::name)
.sorted().forEach(System.out::println);
RandomGenerator rng = RandomGeneratorFactory.of("Xoshiro256PlusPlus").create();
```

5. Java 21: Virtual Threads & Structured Concurrency

Java 21 (LTS) delivers **Virtual Threads** (Project Loom) — lightweight threads managed by the JVM rather than the OS. They allow writing simple blocking code with massive concurrency. **Structured Concurrency** (preview) treats groups of tasks as a unit with clean lifecycle management.

```
// Virtual Threads — Java 21
Thread vt = Thread.ofVirtual().start(() -> {
System.out.println("Running in virtual thread: " + Thread.currentThread());
});
// ExecutorService with virtual threads
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
IntStream.range(0, 100_000).forEach(i ->
executor.submit(() -> {
Thread.sleep(Duration.ofSeconds(1));
return i;
})
);
}
// Structured Concurrency (Java 21 preview)
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
Future<String> user = scope.fork(() -> fetchUser(id));
Future<String> account = scope.fork(() -> fetchAccount(id));
scope.join().throwIfFailed();
return new UserAccount(user.get(), account.get());
}
// Platform thread vs Virtual thread
Thread platform = Thread.ofPlatform().start(() -> {/* 1:1 OS thread */});
Thread virtual = Thread.ofVirtual().start(() -> {/* JVM-managed, millions possible */});
```

Aspect	Platform Thread	Virtual Thread
Backing	OS thread (1:1)	JVM-managed (M:N on carrier threads)
Stack size	~1 MB default	~few KB initially, grows dynamically
Scalability	Thousands	Millions
Blocking	Blocks OS thread	Unmounts carrier thread, remounts when ready

Best for	CPU-bound tasks	I/O-bound / high-concurrency tasks
Thread.sleep()	Wastes OS thread	Yields carrier thread — efficient

6. Java 21: Pattern Matching Switch & Sequenced Collections

Pattern Matching for Switch finalised in Java 21 enables exhaustive type-based dispatch with guards. **Sequenced Collections** introduce new interfaces SequencedCollection, SequencedSet, and SequencedMap providing uniform first/last element access.

```
// Pattern Matching Switch (Java 21)
static String format(Object obj) {
return switch (obj) {
case Integer i when i < 0 -> "Negative: " + i;
case Integer i -> "Integer: " + i;
case String s when s.isEmpty() -> "(empty string)";
case String s -> "String of length " + s.length();
case null -> "null value";
default -> "Other: " + obj;
};
}
// Guarded patterns with &&
static double process(Shape shape) {
return switch (shape) {
case Circle c when c.radius() > 100 -> Double.MAX_VALUE; // large
case Circle c -> Math.PI * c.radius() * c.radius();
case Rectangle r -> r.w() * r.h();
};
}
// Sequenced Collections (Java 21)
SequencedCollection<String> list = new ArrayList<>(List.of("a","b","c"));
list.getFirst(); // "a"
list.getLast(); // "c"
list.addFirst("z"); // prepend
list.reversed(); // reversed view
SequencedMap<String, Integer> map = new LinkedHashMap<>();
map.firstEntry(); // Map.Entry<"a", 1>
map.lastEntry();
map.reversed(); // reversed view
```

7. Modern Java Feature Timeline (Java 9–21)

Version	Key Features	Type
Java 9 (2017)	JPMS modules, JShell, Collection factories, Stream.takeWhile/dropWhile, Optional.stream	Non-LTS
Java 10 (2018)	var (local-variable type inference), App Class-Data Sharing	Non-LTS
Java 11 (2018)	HTTP Client (standard), String::strip/isBlank/lines, var in lambda, LTS	LTS
Java 12 (2019)	Switch expressions (preview), Shenandoah GC	Non-LTS
Java 13 (2019)	Text Blocks (preview), Dynamic CDS Archives	Non-LTS
Java 14 (2020)	Records (preview), Pattern instanceof (preview), Helpful NPE messages	Non-LTS
Java 15 (2020)	Text Blocks (standard), Sealed Classes (preview), Hidden Classes	Non-LTS
Java 16 (2021)	Records (standard), Pattern instanceof (standard), Stream.toList()	Non-LTS

Java 17 (2021)	Sealed Classes (standard), Enhanced PRNG, Remove SecurityManager, LTS	LTS
Java 18 (2022)	UTF-8 by default, Simple Web Server, Code Snippets in Javadoc	Non-LTS
Java 19 (2022)	Virtual Threads (preview), Structured Concurrency (incubator)	Non-LTS
Java 20 (2023)	Scoped Values (incubator), Record Patterns (preview)	Non-LTS
Java 21 (2023)	Virtual Threads, Sequenced Collections, Pattern Switch, Record Patterns, LTS	LTS

8. GraalVM, Project Panama & Future Directions

Beyond language features, the JVM ecosystem continues to evolve with **GraalVM Native Image** for ahead-of-time compilation, **Project Panama** (Foreign Function & Memory API in Java 21) for native interop, and **Project Valhalla** (value types) on the horizon.

```
// Foreign Function & Memory API (Java 21 preview / Panama)
try (Arena arena = Arena.ofConfined()) {
MemorySegment segment = arena.allocate(1024); // native memory
MemorySegment str = arena.allocateFrom("Hello from native!");
// Call native function via MethodHandle
Linker linker = Linker.nativeLinker();
SymbolLookup stdlib = linker.defaultLookup();
MethodHandle strlen = linker.downcallHandle(
stdlib.find("strlen").orElseThrow(),
FunctionDescriptor.of(JAVA_LONG, ADDRESS)
);
long len = (long) strlen.invoke(str);
}
// Record Patterns (Java 21)
record Pair<A,B>(A first, B second) {}
Object obj = new Pair<>("hello", 42);
if (obj instanceof Pair(String s, Integer i)) {
System.out.println(s + " " + i);
}
// Scoped Values (Java 21 preview) — safer ThreadLocal alternative
static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();
ScopedValue.where(CURRENT_USER, "alice").run(() -> {
System.out.println(CURRENT_USER.get()); // "alice"
});
```

GraalVM Native Image

Native Image compiles Java to a standalone native executable with no JVM at runtime. Benefits: instant startup (~10ms), lower memory footprint. Tradeoffs: no dynamic class loading, reflection must be declared, no JIT optimisations. Used heavily in Quarkus and Micronaut for serverless/container workloads. Requires native-image tool from GraalVM distribution.

Interview Q&A; — Standard Questions

Q1. What problem does JPMS (Java 9 Module System) solve?
JPMS solves classpath hell, encapsulation leaks, and unreliable configuration in large applications. Before modules, all JARs were on the flat classpath — any class could access any other class. Modules enforce strong encapsulation (only exported packages are accessible), reliable configuration (missing/duplicate modules are detected at startup), and allow the JDK itself to be modularised and stripped down.

Q2. What is the difference between 'exports' and 'opens' in module-info.java?
exports makes a package accessible for compile-time and runtime use by other modules — but not for deep reflection. **opens** allows deep reflection (e.g., private field access) at runtime via java.lang.reflect or java.lang.invoke, without making it usable for normal compilation. Spring and Hibernate rely on 'opens' to inject fields and proxy classes.

Q3. What does 'var' do in Java 10, and what are its limitations?
var enables local-variable type inference — the compiler infers the static type from the right-hand side expression. Limitations: (1) only for local variables (not fields, method params, return types), (2) initializer is mandatory, (3) cannot be null (compiler can't infer type from null), (4) not dynamic — once inferred, the type is fixed. 'var' is not a keyword but a reserved type name.

Q4. What new String methods were added in Java 11?
Java 11 added: **strip()** / stripLeading() / stripTrailing() — Unicode-aware whitespace removal (unlike trim() which uses ASCII). **isBlank()** — true if empty or only whitespace. **lines()** — returns Stream<String> of lines. **repeat(n)** — repeats string n times. Java 12 added indent(n) and transform(f). Java 15 added formatted() (String.format shortcut).

Q5. What is a Record in Java 16, and what does it auto-generate?
A **record** is an immutable data carrier class. 'record Point(double x, double y) {}' auto-generates: (1) private final fields x and y, (2) canonical constructor Point(double x, double y), (3) accessor methods x() and y(), (4) equals() comparing all components, (5) hashCode() based on all components, (6) toString() like 'Point[x=1.0, y=2.0]'. Records are implicitly final and cannot extend other classes.

Q6. What is a sealed class and why is it useful?
A **sealed class/interface** restricts which classes can extend or implement it, declared with 'sealed … permits SubA, SubB'. Permitted subclasses must be in the same package/module and must be final, sealed, or non-sealed. Benefits: (1) exhaustive pattern matching — compiler knows all subtypes, (2) domain modelling (algebraic data types), (3) security — prevents unintended extension.

Q7. What is a Text Block in Java 15 and how does it handle indentation?
A Text Block is a multi-line string literal delimited by triple quotes. The compiler automatically strips **incidental leading whitespace** based on the least-indented non-blank line. The closing delimiter position controls how much whitespace is stripped. Re-indent() and stripIndent() methods also available. Escape sequences like \s (trailing space) and \ (line continuation) are supported.

Q8. What are Virtual Threads in Java 21 and how do they differ from platform threads?
Virtual threads are lightweight threads managed by the JVM (Project Loom). They are mounted on carrier (platform) threads when running and unmounted when blocking (I/O, sleep, lock). Unlike platform threads (1:1 OS threads, ~1MB stack), millions of virtual threads can coexist with minimal memory. They use the same Thread API, so no code changes needed for blocking code. Best suited for high-concurrency I/O-bound workloads.

Q9. What are Sequenced Collections in Java 21?

Java 21 introduced three new interfaces: SequencedCollection (adds getFirst/getLast/addFirst/addLast/reversed), SequencedSet (extends both SequencedCollection and Set), and SequencedMap (adds firstEntry/lastEntry/reversed). These are retrofitted onto existing classes: List, Deque, LinkedHashSet implement SequencedCollection; LinkedHashMap implements SequencedMap. Solves the inconsistency of first/last access across collections.

Q10. What is Pattern Matching for switch in Java 21?

Pattern matching switch allows matching on the type and structure of an expression: 'switch(obj) { case String s -> … case Integer i when i > 0 -> … case null -> … }'. Key features: (1) type patterns with binding variables, (2) guarded patterns using 'when', (3) null handling in case labels, (4) exhaustiveness checking for sealed types, (5) dominance rules — more specific patterns must appear before general ones.

Q11. What are the Collection factory methods added in Java 9?

Java 9 added: **List.of(e1, e2, …)**, **Set.of(e1, e2, …)**, **Map.of(k1,v1, k2,v2, …)** (up to 10 pairs), and **Map.ofEntries(Map.entry(k,v), …)** for more. These are **immutable** (throw UnsupportedOperationException on mutation), null-hostile (throw NullPointerException), and not Serializable. Set.of and Map.of keys/values must be distinct.

Q12. What Stream methods were added in Java 9?

Java 9 added: **takeWhile(predicate)** — takes elements while predicate is true, then stops; **dropWhile(predicate)** — drops elements while predicate is true, then passes remaining; **iterate(seed, predicate, next)** — finite iterate with a stop condition; **ofNullable(t)** — empty stream if null, singleton otherwise. Java 16 added Stream.toList() as a shortcut for collect(Collectors.toUnmodifiableList()).

Q13. What is Structured Concurrency in Java 21?

Structured Concurrency (preview) treats a group of concurrent tasks as a single unit of work. StructuredTaskScope ensures: if a child task fails, siblings are cancelled (ShutdownOnFailure); or succeeds when the first child succeeds (ShutdownOnSuccess). The scope must be joined before results are accessed. This prevents thread leaks and simplifies error handling compared to raw CompletableFuture or ExecutorService.

Q14. What are Scoped Values in Java 21?

Scoped Values (preview) are a safer, immutable alternative to ThreadLocal, designed for virtual threads. A ScopedValue is bound for a bounded scope: 'ScopedValue.where(USER, "alice").run(() -> { … USER.get() … })'. Unlike ThreadLocal: (1) immutable within scope, (2) inherited by child virtual threads automatically, (3) no risk of memory leaks from forgetting remove(), (4) better performance at scale.

Q15. What helpful NullPointerException messages were added in Java 14?

Java 14 added **Helpful NPEs** (enabled by default from Java 15+). Instead of just 'NullPointerException', the message now says which variable was null: 'Cannot invoke "String.length()" because "str" is null' or 'Cannot store to int array because "arr" is null'. Enable explicitly in earlier JVMs with: -XX:+ShowCodeDetailsInExceptionMessages.

Q16. What is the difference between LTS and non-LTS Java releases?

Java releases every 6 months. **LTS (Long-Term Support)** versions (Java 8, 11, 17, 21) receive extended support and security patches for several years and are recommended for production. **Non-LTS** versions receive updates only until the next release (6 months). Enterprises typically run LTS releases; developers may explore non-LTS features. Java 25 (September 2025) will be the next LTS.

Q17. What is the Foreign Function & Memory API in Java 21?

The FF&M API (Project Panama, Java 21 preview) allows Java to call native libraries and manage off-heap memory safely. Key classes: **Arena** (manages native memory lifetime), **MemorySegment** (contiguous memory region), **Linker** (bridges Java and native ABI), **MethodHandle** (native function reference). Replaces unsafe sun.misc.Unsafe and JNI with a safe, garbage-collected approach.

Q18. What is GraalVM Native Image?

GraalVM Native Image compiles Java applications ahead-of-time (AOT) into standalone native executables using the Substrate VM. Benefits: near-instant startup (no JVM warmup), low memory footprint. Limitations: no dynamic class loading at runtime, reflection must be declared in config files, no JIT profiling/optimisation. Used by frameworks like Quarkus, Micronaut, and Spring Native for microservices and serverless deployments.

Q19. What are Record Patterns in Java 21?

Record Patterns enable deconstruction of records in instanceof and switch: 'if (obj instanceof Pair(String s, Integer i))'. The pattern matches if obj is a Pair AND the first component is a String AND the second is an Integer, binding s and i. Nested patterns are supported: 'case Pair(Point(var x, var y), Color c)'. Combined with sealed types this enables exhaustive, type-safe data processing.

Q20. What Java 9 changes were made to Optional?

Java 9 added three methods to Optional: (1) **ifPresentOrElse(consumer, runnable)** — perform action if present, else run empty-action; (2) **or(supplier)** — return same Optional if present, else return another Optional from supplier (vs orElseGet which returns T); (3) **stream()** — returns Stream of one element if present, empty stream if empty. Useful for flatMapping optionals: optionals.stream().flatMap(Optional::stream).

Q21. What is Switch Expressions (Java 14) vs traditional switch?

Switch expressions (finalized Java 14) differ from traditional switch: (1) they return a value and can be used in assignments; (2) arrow syntax '-> value/block' eliminates fall-through; (3) 'yield' statement returns a value from a block case; (4) exhaustiveness required when used as expression (compiler enforces default or covers all cases). 'var result = switch(day) { case MON, TUE -> 1; case WED -> { yield compute(); } default -> 0; }'

Q22. What is Stream.toList() added in Java 16?

Java 16 added Stream.toList() as a terminal operation that collects stream elements into an **unmodifiable List**. It is a shorthand for collect(Collectors.toUnmodifiableList()). The returned list preserves encounter order and throws UnsupportedOperationException on mutation. Unlike Collectors.toList() which returns a mutable list (implementation-defined, usually ArrayList).

Q23. What improvements were made to instanceof pattern matching in Java 16?

'instanceof' pattern matching (Java 16 standard) binds the cast variable in the if-block scope: 'if (obj instanceof String s) { s.length(); }'. The pattern variable s is in scope only when the condition is true. **Negative patterns**: 'if (!(obj instanceof String s)) return; s.length();' also works — s is in scope after the early return. Guards: 'obj instanceof String s && s.length() > 5'.

Q24. What is the HTTP Client API in Java 11?

The java.net.http.HttpClient (Java 11) supports HTTP/1.1 and HTTP/2, WebSockets, and both synchronous and asynchronous requests. Key classes: HttpClient (built via builder with version, timeout, authenticator, proxy, SSL), HttpRequest (URI, method, headers, body publisher), HttpResponse (status, headers, body handler). Async: client.sendAsync() returns CompletableFuture. Replaces the old HttpURLConnection API.

Q25. What is App Class-Data Sharing (CDS) in Java 10?

Class-Data Sharing (CDS, Java 10) allows sharing a class-metadata archive across JVM instances. The JVM dumps loaded class metadata to a shared archive (.jsa file) at first run. Subsequent runs memory-map the archive, reducing startup time and memory footprint. Java 13 added Dynamic CDS (auto archive on app exit). Java 17 made the default CDS archive always-on. Particularly useful for microservices with many short-lived JVM instances.

Q26. What are the key changes to Process API in Java 9?
Java 9 enhanced java.lang.Process and added ProcessHandle: ProcessHandle provides PID, info (command, arguments, start time, CPU time), descendants(), parent(), and onExit() (CompletableFuture). ProcessBuilder.startPipeline() chains processes like shell pipes. Example: ProcessHandle.current().info().command() gives the JVM executable path. Useful for monitoring, managing, and building process pipelines from Java.

Interview Q&A; — Advanced Questions

[ADV] Q27. How does the JVM implement Virtual Thread mounting/unmounting under the hood?
Virtual threads run on carrier (platform) threads from a ForkJoinPool. When a virtual thread blocks (I/O, Object.wait, Lock.lock), the JVM performs a **continuation yield**: the virtual thread's stack is captured as a Continuation object on the heap, freeing the carrier thread. When the blocking operation completes, the virtual thread is scheduled on any available carrier and its continuation is resumed. Synchronized blocks currently **pin** the carrier thread (fixed in Java 24+). Use ReentrantLock to avoid pinning.

[ADV] Q28. Explain the JPMS split package problem and how it breaks modules.
A **split package** occurs when the same package name appears in two different modules. JPMS forbids this — the module system cannot determine from which module to load a class in a split package. Common culprit: java.xml.ws split between the JDK and a third-party JAR. Solutions: (1) shade (repackage) conflicting dependencies, (2) use the unnamed module (classpath), (3) use --patch-module to merge packages. Split packages were allowed on the classpath but are a hard error in the module system.

[ADV] Q29. How does the JVM's Continuation mechanism enable virtual threads?
Java 21's virtual threads are built on **Continuations** (java.lang.Continuation, internal). A Continuation captures the entire call stack of a computation, allows it to yield (suspend), and later resume from the same point. Virtual threads wrap a Continuation — when blocked, yield() saves the stack to the heap; run() resumes it. This is cooperative (not preemptive) scheduling at the JVM level. The ForkJoinPool scheduler decides when to mount/unmount.

[ADV] Q30. What is the performance implication of using var vs explicit types?
At the bytecode level, **var has zero performance impact** — the compiler infers the type at compile time and emits identical bytecode to an explicit type declaration. Performance concerns with var are purely about readability and maintenance. Potential pitfall: var with diamond operator 'var list = new ArrayList<>()' infers ArrayList (raw-ish) not List — may expose implementation type and reduce flexibility when refactoring. Always use var with readable RHS.

[ADV] Q31. How does pattern matching switch handle exhaustiveness checking with sealed types?
When switching on a sealed type, the compiler performs **exhaustiveness analysis**: it enumerates all permitted subtypes and verifies every case is covered. If a subtype is missing, a compile error occurs (no default needed). With guards ('when'), exhaustiveness is conservative — a guarded case does not satisfy exhaustiveness alone (the guard might be false). Adding a new permitted subtype to a sealed hierarchy breaks all exhaustive switches — a deliberate design choice to force callers to handle new cases.

[ADV] Q32. What is pinning in virtual threads and how do you avoid it?
A virtual thread is **pinned** to its carrier platform thread when inside a synchronized block/method or when calling native code. Pinning means the carrier thread cannot be unmounted, eliminating the scalability benefit. Diagnosis: use -Djdk.tracePinnedThreads=full or JFR VirtualThreadPinned event. Fix: replace synchronized with ReentrantLock or other java.util.concurrent locks, which support virtual thread unmounting during blocking.

[ADV] Q33. How does the Foreign Function & Memory API compare to JNI?

JNI requires C header files, native stub compilation, UnsafeMemory access, and is error-prone. FF&M API (Project Panama): (1) pure Java — no C code needed for most native calls; (2) Arena-managed memory — automatic cleanup without manual free(); (3) MethodHandle-based — type-safe and JIT-optimizable; (4) VarHandle for struct field access. Compared to JNI, FF&M is safer (bounds-checked), more performant (no GC pinning for off-heap), and easier to use.

[ADV] Q34. What is the difference between StructuredTaskScope.ShutdownOnFailure and ShutdownOnSuccess?

ShutdownOnFailure: when any child task fails, all remaining tasks are cancelled. scope.join().throwIfFailed() re-throws the first failure. Used when all results are needed (e.g., gather user + account data — fail fast if either fails). **ShutdownOnSuccess**: when any child task succeeds, all remaining tasks are cancelled. scope.result() returns the first successful result. Used for racing alternatives (e.g., try two cache servers, use whichever responds first).

[ADV] Q35. Explain Class-Data Sharing (CDS) and how it improves startup time.

CDS works by serializing class metadata (class hierarchy, method bytecodes, string pool) into a memory-mapped archive. On subsequent JVM startups, the archive is mmap'd into the JVM process — the OS can share the physical pages across multiple JVM processes (CoW). This skips class parsing and verification, reducing startup by 10-50% and RSS by 100-300MB for typical Spring Boot apps. Java 19+ added **Dynamic CDS** (automatic archiving on exit). Combined with AOT (GraalVM) for maximum startup reduction.

[ADV] Q36. How does type inference for var interact with generic types and wildcards?

var infers the **most specific type** from the RHS expression. 'var list = new ArrayList<>()' infers ArrayList<Object> (erased), not List or ArrayList<String>. 'var nums = List.of(1,2,3)' infers List<Integer>. Wildcards: 'var x = getWildcard()' where getWildcard returns List<?> infers List<?> — x.add() is then forbidden. Intersection types from lambdas: if var captures a lambda, the type is the function interface, not the intersection type.

[ADV] Q37. What are the memory model implications of virtual thread handoff between carriers?

When a virtual thread is unmounted from carrier A and remounted on carrier B, the JVM inserts **happens-before edges** at mount/unmount points — ensuring the virtual thread sees all writes made before unmounting, and all writes made on the new carrier before mounting are visible. This is specified in the Java Memory Model extension for virtual threads. Effectively, a virtual thread has a consistent view of its own state regardless of which carrier it runs on.

[ADV] Q38. How does GraalVM's SubstrateVM handle reflection in native images?

SubstrateVM performs **closed-world analysis** — all code reachable at runtime must be known at build time. Dynamic reflection, proxy generation, and serialization require configuration. GraalVM agent: run app with '-agentlib:native-image-agent=config-output-dir=META-INF/native-image' to auto-generate reflect-config.json, proxy-config.json, serialization-config.json. Frameworks like Quarkus/Micronaut generate these at build time via annotation processing. Missing config causes NoSuchMethodException or ClassNotFoundException at native runtime.

[ADV] Q39. What is the relationship between Records and Java serialization?

Records are serializable if they implement Serializable. The serialization mechanism for records uses the **canonical constructor** (not the usual readObject/readResolve). This means: (1) serialization cannot bypass validation in the canonical constructor; (2) all record components must be serializable; (3) serialVersionUID is optional (computed from component types). This fixes a long-standing Java serialization vulnerability — records cannot have inconsistent state injected via deserialization.

[ADV] Q40. How does the JVM support multiple concurrent versions of modules?
JPMS does **not** natively support multiple versions of the same module simultaneously in one layer. However, **module layers** (ModuleLayer) allow loading a module in a child layer with a different version — each layer has its own classloader. OSGi solves this with its own class-loading framework. Maven shading (repackaging) is the pragmatic solution for dependency conflicts. The unnamed module (classpath) and automatic modules provide backward compatibility for non-modularised JARs.

[ADV] Q41. What are the performance characteristics of Text Blocks vs String concatenation?
Text Blocks are a compile-time feature — they are compiled to regular String literals with processed indentation and escape sequences. At runtime, a text block is just a String constant in the constant pool — identical performance to a string literal. There is no runtime processing overhead. String.formatted() (called on a text block) is equivalent to String.format() and does involve runtime formatting cost. The indentation stripping happens at compile time, not runtime.

[ADV] Q42. Explain how Stream.takeWhile differs from filter when used with ordered streams.
takeWhile is a **short-circuiting stateful intermediate operation**: it takes elements sequentially until the predicate fails, then stops processing — subsequent elements matching the predicate are NOT included. **filter** evaluates the predicate on every element independently regardless of order. Example: Stream.of(2,4,6,5,8).takeWhile(n -> n%2==0) → [2,4,6] (stops at 5). filter(n -> n%2==0) → [2,4,6,8] (evaluates all). For parallel streams, takeWhile is more expensive (must respect encounter order).

[ADV] Q43. What is a Hidden Class in Java 15 and what is it used for?
Hidden classes are classes that cannot be used directly by the bytecode of other classes and are intended for use by frameworks that generate classes at runtime. Created via Lookup.defineHiddenClass(). Key properties: (1) not discoverable by name in ClassLoader; (2) can be unloaded independently of their defining class loader; (3) shorter lifecycle — GC'd when no references remain. Used by: lambda implementation (replaces anonymous classes generated by LambdaMetafactory), byte-buddy, ASM for dynamic proxy generation.

[ADV] Q44. How does JEP 425 (Virtual Threads) interact with ThreadLocal?
ThreadLocal works with virtual threads — each virtual thread has its own ThreadLocal storage. The problem: if a thread pool reuses platform threads, ThreadLocal state can leak between tasks. With virtual-thread-per-task model, each task gets a fresh virtual thread — ThreadLocal is isolated per task. However, ThreadLocal still allocates a Map per virtual thread, which can be expensive at millions of threads. The preferred solution for virtual threads is **Scoped Values** — immutable, inherited, and lower memory overhead.

[ADV] Q45. What is the significance of exhaustiveness in sealed-type pattern switches for API design?
Sealed types + exhaustive switches create a **visitor pattern built into the language**. When a library adds a new permitted subtype, all exhaustive switches in client code break at compile time — forcing clients to handle the new case. This is intentional: it makes APIs evolvable with compile-time safety rather than runtime surprises. Compare to interface + instanceOf chains which silently ignore new subtypes. For internal APIs or libraries, this is a major advantage over non-sealed hierarchies.

[ADV] Q46. How does the JVM optimise records at the JIT level?
Records benefit from several JIT optimisations: (1) **Scalar replacement** — small records allocated on a hot path may be decomposed into fields and stack-allocated rather than heap-allocated; (2) **Inlining** — accessor methods (x(), y()) are trivial and always inlined; (3) **Escape analysis** — if a record does not escape a method, it may never be heap-allocated. Project Valhalla (future) will introduce value types — records without object identity — enabling even more aggressive flattening.

[ADV] Q47. What makes Optional.or() different from Optional.orElseGet() in Java 9?
orElseGet(Supplier<T>) returns a plain value T if the Optional is empty. **or(Supplier<Optional<T>>)** (Java 9) returns another Optional<T> if empty. This enables chaining: opt.or(() -> cache.get(key)).or(() -> db.find(key)).orElseThrow(). Without or(), you would need nested ifPresent checks or flatMap tricks. or() preserves the Optional wrapper, allowing further Optional transformations in the chain.

[ADV] Q48. Explain the Memory Segment and Arena model in the Foreign Memory API.
A **MemorySegment** is a contiguous region of memory (on-heap or off-heap) with a spatial bound (size) and a temporal bound (lifetime controlled by Arena). **Arena** types: confined (single-thread access, deterministic close), shared (multi-thread), auto (GC-managed, non-deterministic), global (never closed). Accessing a MemorySegment after its Arena is closed throws IllegalStateException. VarHandle or MemoryLayout provides typed access to struct fields. All accesses are bounds-checked — no segfaults from Java code.

[ADV] Q49. How does JShell implement incremental compilation and state?
JShell maintains a **state machine** — each snippet (expression, statement, declaration, import) is wrapped in a synthetic class and compiled incrementally. When a snippet redefines a variable or method, JShell replaces the previous snippet and recompiles dependents. Snippets are executed via class loading into a remote JVM (execution engine can be configured). State (variables, methods, imports) persists across snippets. JShell API (javax.tools.JavaCompiler-based) allows embedding REPL functionality in tools.

[ADV] Q50. What are the implications of removing SecurityManager in Java 17?
SecurityManager was deprecated in Java 17 (JEP 411) and removed in Java 24. It provided a policy-based sandboxing mechanism, but was rarely configured correctly, had performance overhead on every privileged operation, and did not work with modern container-based isolation. Replacements: OS-level sandboxing (seccomp, AppArmor, containers), module encapsulation (JPMS strong encapsulation), Java agent-based monitoring. Libraries relying on AccessController.doPrivileged() need migration.

[ADV] Q51. How does dynamic CDS (Class Data Sharing) work with Spring Boot?
Spring Boot 3.x supports CDS natively. Process: (1) 'java -Dspring.context.exit=onRefresh -jar app.jar' performs a training run (starts, initialises context, dumps archive); (2) subsequent runs: 'java -XX:SharedArchiveFile=app.jsa -jar app.jar' use the archive. The CDS archive includes Spring's class metadata, parsed bytecodes, and string pools. Combined with checkpoint/restore (CRaC — Coordinated Restore at Checkpoint, Project CRaC), Spring Boot apps can achieve sub-100ms startup times.

[ADV] Q52. What is the difference between value-based classes and value types (Project Valhalla)?
Value-based classes (e.g., Optional, LocalDate, Integer wrappers) are existing classes where identity operations (==, synchronize, identityHashCode) are discouraged/undefined — they are flagged with @jdk.internal.ValueBased. **Value types** (Project Valhalla, Java 25+): primitive-like objects flattened in memory arrays/fields with no object header, no identity. Declared with 'value class'. Allow JVM to store them inline in arrays and fields — eliminating indirection and GC pressure for small immutable aggregates like Point, Range, Complex.

Coding Exercises

Exercise 1: Sealed Shape Hierarchy with Exhaustive Switch

Define a sealed interface Shape with records Circle, Rectangle, Triangle as permitted types. Write a method 'double area(Shape s)' using an exhaustive pattern-matching switch. Ensure adding a new Shape causes a compile error.

```
public sealed interface Shape permits Circle, Rectangle, Triangle {}
public record Circle(double radius) implements Shape {}
public record Rectangle(double width, double height) implements Shape {}
public record Triangle(double base, double height) implements Shape {}
public class ShapeCalculator {
public static double area(Shape s) {
return switch (s) {
case Circle c -> Math.PI * c.radius() * c.radius();
case Rectangle r -> r.width() * r.height();
case Triangle t -> 0.5 * t.base() * t.height();
// No default needed — compiler verifies exhaustiveness
};
}
public static void main(String[] args) {
System.out.println(area(new Circle(5))); // ~78.5
System.out.println(area(new Rectangle(4, 6))); // 24.0
}
}
```

Exercise 2: Virtual Thread HTTP Fetch with Structured Concurrency

Use StructuredTaskScope.ShutdownOnFailure to fetch two URLs concurrently using virtual threads, combining results. Handle failure with throwIfFailed.

```
import java.net.http.*;
import java.util.concurrent.StructuredTaskScope;
record FetchResult(String url, int status) {}
public class ConcurrentFetcher {
private final HttpClient http = HttpClient.newHttpClient();
public record Report(FetchResult r1, FetchResult r2) {}
public Report fetchBoth(String url1, String url2) throws Exception {
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
var f1 = scope.fork(() -> fetch(url1));
var f2 = scope.fork(() -> fetch(url2));
scope.join().throwIfFailed();
return new Report(f1.get(), f2.get());
}
}
private FetchResult fetch(String url) throws Exception {
var req = HttpRequest.newBuilder().uri(URI.create(url)).GET().build();
var resp = http.send(req, HttpResponse.BodyHandlers.discarding());
return new FetchResult(url, resp.statusCode());
}
}
```

Exercise 3: Text Block JSON Builder with Records

Write a method that takes a list of Person records and returns a JSON array string using text blocks.

```
record Person(String name, int age, String email) {}
public class JsonBuilder {
public static String toJson(List<Person> people) {
```

```
var entries = people.stream()
.map(p -> """
{
"name": "%s",
"age": %d,
"email": "%s"
}""".formatted(p.name(), p.age(), p.email()))
.collect(Collectors.joining(",\n"));
return """
[
%s
]""".formatted(entries);
}
}
```

Exercise 4: Module-aware ServiceLoader with JPMS

Define a GreeterService SPI, two implementations, and wire them via module-info.java using uses/provides directives and ServiceLoader.

```
// module-info.java (api module)
module com.example.api {
exports com.example.api;
uses com.example.api.GreeterService;
}
// GreeterService.java
package com.example.api;
public interface GreeterService {
String greet(String name);
}
// module-info.java (impl module)
module com.example.impl {
requires com.example.api;
provides com.example.api.GreeterService
with com.example.impl.EnglishGreeter,
com.example.impl.FrenchGreeter;
}
// Main usage
ServiceLoader<GreeterService> loader =
ServiceLoader.load(GreeterService.class);
loader.forEach(s -> System.out.println(s.greet("World")));
```

Exercise 5: Sequenced Collections — Deque as Sliding Window

Use SequencedCollection (ArrayDeque) to maintain a sliding window of the last N integers, computing the running average.

```
public class SlidingWindowAverage {
private final ArrayDeque<Integer> window;
private final int capacity;
private long sum = 0;
public SlidingWindowAverage(int capacity) {
this.capacity = capacity;
this.window = new ArrayDeque<>(capacity);
}
public double add(int value) {
if (window.size() == capacity) {
sum -= window.removeFirst(); // SequencedCollection method
}
window.addLast(value); // SequencedCollection method
```

```
sum += value;
return (double) sum / window.size();
}
public static void main(String[] args) {
var avg = new SlidingWindowAverage(3);
System.out.println(avg.add(1)); // 1.0
System.out.println(avg.add(2)); // 1.5
System.out.println(avg.add(3)); // 2.0
System.out.println(avg.add(7)); // 4.0 (drops 1)
}
}
```

Debug & Fix Scenarios

Debug 1: Virtual Thread Pinning Causing Starvation

```
// BUG: synchronized pins carrier thread, blocking other virtual threads
public class ConnectionPool {
private final Queue<Connection> pool = new LinkedList<>();
public synchronized Connection borrow() {
while (pool.isEmpty()) {
Thread.sleep(10); // BUG: pinned! other virtual threads can't run
}
return pool.poll();
}
}
// FIX: Use ReentrantLock to allow virtual thread unmounting
private final ReentrantLock lock = new ReentrantLock();
private final Condition notEmpty = lock.newCondition();
public Connection borrow() throws InterruptedException {
lock.lock();
try {
while (pool.isEmpty()) notEmpty.await(); // unmounts virtual thread
return pool.poll();
} finally { lock.unlock(); }
}
```

Debug 2: Record Canonical Constructor Bypass via Reflection

```
// BUG: Trying to set a record field via reflection
record Temperature(double celsius) {
Temperature { if (celsius < -273.15) throw new IllegalArgumentException(); }
}
// This will FAIL — record components are final
Temperature t = new Temperature(25.0);
Field f = Temperature.class.getDeclaredField("celsius");
f.setAccessible(true);
f.setDouble(t, -999); // throws IllegalAccessException — records are immutable
// CORRECT: You cannot mutate records — create a new instance instead
Temperature adjusted = new Temperature(t.celsius() + 5);
```

Debug 3: JPMS Missing 'opens' Causing Reflection Failure

```
// BUG: Spring injection fails with InaccessibleObjectException
// java.lang.reflect.InaccessibleObjectException:
// Unable to make private com.app.service.UserService() accessible
// because module com.app does not 'opens com.app.service' to spring.core
// module-info.java BEFORE (broken):
module com.app {
requires spring.core;
exports com.app.api;
// MISSING: no 'opens' for Spring to inject into private fields
}
// FIXED:
module com.app {
requires spring.core;
exports com.app.api;
opens com.app.service to spring.core; // Allow deep reflection
}
```

Debug 4: Text Block Indentation Pitfall

```
// BUG: unexpected leading whitespace in text block
```

```
String sql = """
SELECT *
FROM users
WHERE active = true
""";
// Result has NO leading spaces — correct! Closing delimiter at same indent level

// BUG: closing delimiter at column 0 retains all leading whitespace
String broken = """
SELECT *
FROM users
"""; // closing triple-quote at column 0
// Result: each line has 4 leading spaces - probably not wanted

// FIX: align closing delimiter with content or use stripIndent()
String fixed = """ SELECT *
FROM users
WHERE active = true """.stripIndent();
```

Multiple Choice Questions

Q1. Which directive in module-info.java allows deep reflection at runtime?

A) exports

B) requires

C) opens ✓

D) provides

Q2. What does var infer when used with 'var x = new ArrayList<>()'?

A) List

B) ArrayList ✓

C) Collection

D) Iterable

Q3. Which Java version finalised Records?

A) Java 14

B) Java 15

C) Java 16 ✓

D) Java 17

Q4. What causes a virtual thread to be 'pinned' to its carrier?

A) Thread.sleep()

B) synchronized block ✓

C) CompletableFuture.get()

D) ReentrantLock.lock()

Q5. Which new interface in Java 21 adds getFirst()/getLast() to List?

A) IndexedList

B) OrderedCollection

C) SequencedCollection ✓

D) SortedCollection

Q6. What does StructuredTaskScope.ShutdownOnSuccess do?

A) Cancels all tasks if one fails

B) Returns when all tasks complete

C) Cancels remaining tasks when first task succeeds ✓

D) Retries failed tasks

Quick Reference Cheat Sheet

Feature	Syntax / Key Detail	Version
JPMS Module	module x { requires y; exports p; opens p to m; }	Java 9
Collection Factories	List.of(), Set.of(), Map.of() — immutable, null-hostile	Java 9
Stream.takeWhile	Stream.takeWhile(pred) — stops at first false, not filter	Java 9
Optional.or()	opt.or(() -> Optional.of(fallback)) — chain optionals	Java 9
var keyword	Local only, compile-time inference, no dynamic typing	Java 10
String.strip()	Unicode-aware trim; isBlank(), lines(), repeat(n)	Java 11
HTTP Client	HttpClient.newBuilder().version(HTTP_2).build()	Java 11
Switch Expression	switch(x) { case A -> val; case B -> { yield v; } }	Java 14
Helpful NPEs	-XX:+ShowCodeDetailsInExceptionMessages (default Java 15+)	Java 14
Text Blocks	"""..."""" — incidental indent stripped at compile time	Java 15
Records	record P(int x, int y) {} — final, immutable, auto-methods	Java 16
instanceof patterns	if (obj instanceof String s && s.length() > 5)	Java 16
Sealed Classes	sealed interface S permits A, B; subclass: final/sealed/non-sealed	Java 17
Stream.toList()	Shortcut for collect(toUnmodifiableList())	Java 16
Virtual Threads	Thread.ofVirtual().start(() -> {}); Executors.newVirtualThreadPerTaskExecutor()	Java 21
Pattern Switch	switch(s) { case Circle c when c.r()>0 -> …; case null -> …; }	Java 21
SequencedCollection	getFirst/getLast/addFirst/addLast/reversed() on List/Deque/LinkedHashMap	Java 21
Scoped Values	ScopedValue.where(K, v).run(() -> K.get()) — immutable ThreadLocal alt	Java 21